Performance Evaluation of Features and Clustering Algorithms for Malware

Houtan Faridi
Department of Computer Science
University of Houston, TX
Email: hfaridi@uh.edu

Srivathsan Srinivasagopalan
Alienvault Inc
Austin, TX
Email: ssrinivasagopalan@alienvault.com

Rakesh Verma
Department of Computer Science
University of Houston, TX
Email: rverma@uh.edu

***Abstract*—Malware has become a huge problem and obfuscation techniques used by attackers means that signature management for malware becomes too time and space consuming. If we can cluster different versions of the same malware together, we can attain many benefits. In this paper, we study the malware clustering problem systematically using several algorithms, distance functions and sets of features. A real-world, ground-truth dataset and multiple metrics are used to evaluate the performance of these suite of approaches. Agglomerative Hierarchical clustering, Spectral clustering, and DBSCAN emerged as the winning algorithms.**

***Index Terms*—Malware, clustering, algorithm, cybersecurity.**

I. Introduction

From its conception in 1971, malware undoubtedly has become a major threat in modern society, and the numbers of infections are increasing daily. With more sensitive information becoming digitalized, incentives[1] have led to a rise in dedicated teams and cyber criminal organizations. The Internet today is increasingly exploited by highly skilled malware developers and has even become home to large black markets for the purchase and spreading of malware. This provides a strong incentive for the malware developers to modify and increase the complexity of their malicious code in order to decrease the chances of being detected by anti-virus programs. By using different obfuscation techniques or constructing multiple permutations of malicious software, authors can ensure other versions of their malware continue to function, even if a signature is developed for one version. This leads to multiple forks or new implementations of the same type of malicious software that can propagate out of control.

Approximately, 400,000 new malware are being registered daily,[2] which gives rise to the problem of processing the huge amount of data obtained from malware analysis. This also makes it challenging for anti-virus vendors to detect zero-day attacks and release updates in a reasonable time-frame to prevent infection and propagation.

Malware researchers and anti-virus companies try to analyze the code by some combination of static and dynamic analysis. A good analysis will help in understanding the malware, creating signatures, and using them to detect and thwart any subsequent attack attempts. Due to increasing sophistication of today's malware that uses polymorphism, metamorphism, and other obfuscation methods, the researchers and companies are facing a huge uphill task.

Hence, to ensure that large fraction of malware is easily analyzed and understood, a common technique is to cluster them into groups of malware that have similar characteristics. Once a population of malware is clustered well, several useful insights can be gleaned from each cluster, such as salient features, lineage among the members and signature generation.

Moreover, when these clusters are monitored over a long period of time, a better understanding of their activities can be easily seen. For example, it will help in visualizing the relationships and their evolution over time. Eg: (a) A variant of X that was more prevalent during the second quarter. (b) Families that were only present during the third quarter of 2016. (c) An XYZ-breed that faded-off before the first-quarter of 2016.

From a productivity perspective, clustering helps in creating one signature for a family of malware (instead of creating one signature for each malware). This has the potential to significantly improve the efficacy and latency in developing a solution that is both broad-enough to detect an entire family of malware and specific-enough to detect a particular instance.

Finally, clustering malware periodically and monitoring them carefully over time will help in quickly identifying anomalies and spotting new clusters. Identifying a new family that has begun to spread in past few hours is a difficult problem if we do not have such pipelines in place. By clustering and monitoring, anti-virus companies could potentially learn the existence (and prevalence) of a new malware family in a relatively short time, detect their salient features, and quickly generate a signature to help thwart this new family. Furthermore, they can disseminate the information they had gleaned about this family to the public using several of the exchanges that share information.

A. Problem Statement

Given a set of documents representing malicious sample behaviors, can we generate a *robust* set of clusters $\mathcal{C} \leftarrow \{c_1, c_2, \ldots, c_j\}$ based on a *configurable* set of features. By *robust*, we insist that the members of different clusters must have considerable differences.

[1]According to a recent report [1], more than $1.1M is lost to cyber-crime every minute.

[2]Some latest malware stats can be found here: [2, 3, 4, 5]

2375-9259/18/$31.00 ©2018 IEEE
DOI 10.1109/ICDMW.2018.00010

B. Contributions

In this work, we provide a detailed, systematic study of clustering malware based on their behavioral characteristics. We examine multiple sets of features including network connections and system interaction performed by malware. Feature categories of varying granularity are construed and tested using several clustering algorithms on a real dataset from industrial partner AlienVault. To the best of our knowledge, this is the first such attempt, not only for experimenting with multiple algorithms, but also, for the varied parameters and distance metrics used. Moreover, we believe our study is the first to include: (i) Affinity Propagation and two forms of Spectral Clustering, and (ii) Two cutoff methods for hierarchical clustering, number of clusters and inconsistency, which have not been used in a malware setting.

II. Related Work

Considerable research has been done in the past on clustering malware including [6, 7, 8, 9, 10, 11]. Bailey *et al.* [6] clustered malware with features collected from system state changes. Using Normalized Compression Distance, samples were clustered via hierarchical clustering. To determine cluster quality, guidelines for an ideal labeling system were constructed. This definition was then used to compare the resulting clusters with other anti-virus labeling systems.

To reduce the time and cost associated with clustering large collections of malware, Bayer *et al.* [7] performed hierarchical clustering using Locality Sensitive Hashing (LSH) to approximate Jaccard distance. Using extracted system calls, network data, and taint analysis, profiles were constructed which represented interactions performed by malware on the OS. They were able to cluster a set of over 75,000 samples in less than three hours. To determine cluster quality, reference clusters were constructed by collecting all samples which had identical labels across a set of anti-virus systems, resulting in 2,568 samples. A high overall quality was achieved using their constructed metrics of precision and recall. However, as pointed out in [12], constructing a reference cluster via means of anti-virus majority voting could result in only clustering samples which are inherently easy to distinguish.

Vadrevu and Perdisci's work in [8] aims to increase classification efficiency via another method, reducing analysis time. By classifying malware during execution, classification time was reduced by 50% on average with less than 0.3% information loss. In order to classify malware, network features were used to construct a Jaccard similarity matrix which was then used in DBSCAN to cluster samples into families.

Mohaisen *et al.* in [9] uses their AutoMal system to extract malicious features via dynamic analysis. Using these features, hierarchical clustering is performed using multiple similarity metrics including Jaccard, Hamming, cosine, and correlation, run on several linkage methods of Average, Centroid, Complete, Median, Single, and Ward. Once the hierarchies were constructed, a distance threshold of 0.7 was used to determine the clusters. They give precision and recall using a manually vetted set of reference clusters.

Rieck *et al.* ([10]) also used Hierarchical clustering to classify malware. Dynamically extracted malware behavior is collected and encoded using a constructed representation called MIST. Complete linkage hierarchical clustering is then performed using prototypes. With a reference cluster constructed via anti-virus majority voting, Precision, Recall, and F-Measure metrics are used to evaluate results.

III. Dataset and Preprocessing

Prior to feeding malware data to any clustering algorithm, data is preprocessed so that it is conducive for consumption by those algorithms. To that effect, the malware data files are filtered to ensure that only the required features are considered for clustering and altered to ensure that the algorithms distinguish the malware samples appropriately by correctly computing the distances between them. The dataset and preprocessing steps are as follows.

A. Dataset

Our dataset consisted of information extracted from 5,673 malicious Windows portable executables provided by Alienvault. To extract behavioral information from these malware samples, they were run through Cuckoo Sandbox [13]. Cuckoo ran each malicious sample for a maximum of three minutes, then generated a json file for every sample containing a variety of information including network communications and system interactions performed by the malware. Through this information, it is possible to extract features which identify how a sample reacts when executed.

To determine clustering quality, a set of ground truth reference clusters was generated.[3] Those set of malwares that triggered a unique Suricata rule[4] (to generate an alert) were considered to belong to a single family. Likewise, several clusters of malware were generated by grouping all the sets of malwares that triggered several unique signatures. Most of these clusters were then manually 'eye-balled' to ensure that the malwares in them indeed belonged to the same family.

This reference clustering consists of 94 clusters, with cluster members ranging from one to 772 samples. Statistics regarding the reference clustering are shown in Table I. While we explore multiple sources of data for feature extraction, the reference clusters were constructed using only network data.[5]

B. Feature Extraction

Assume that a malware sample has n features in totality. A carefully vetted set of features $F \leftarrow \{f_1, f_2, f_3, \ldots, f_k\}$ were identified by domain experts, where $k << n$. F is considered most useful for effectively distinguishing malware. From this *master feature-set* F, we split features into three initial categories of behaviors, network, and Cuckoo signatures.

Each category was then split into more fine-grained features. The behavioral category was made up of files, registry keys,

[3]Courtesy of Alienvault security engineers.

[4]Suricata is a free and open source, mature, fast and robust network threat detection engine. It has its own format for writing rules.

[5]However, this fact was not disclosed to the clustering team until results had been obtained.

TABLE I
SAMPLES PER REFERENCE CLUSTER STATISTICS

Statistic	Samples
Mean	60.35
St-dev	119.03
Var	14168.90
Median	14.5
Mode	3
Max cluster size	772
Min cluster size	1

TABLE II
NUMBER OF FEATURES PER CATEGORY

Category	Feature Count	Category	Feature Count
Behaviors	13	DNS	1
Network	22	Domains	1
Signatures	2	Hosts	2
Files	4	HTTP	6
Keys	4	ICMP	2
Services	2	IRC	1
Commands	1	SMTP	1
API	1	TCP	4
Mutexes	1	UDP	4

services, commands, resolved API, and mutexes subcategories. Networking category was split into: DNS, domains, hosts, HTTP, ICMP, IRC, SMTP, TCP, and UDP subcategories. Finally, the cuckoo signatures category contained handpicked signatures provided by Cuckoo, which identified higher level malware intent based on sequences of identified interactions. For instance "modified_security_center_warnings" signifies that a process attempted to modify or disable security center warnings. The signature category was split based on the high level action labels. The number of features per category is shown in Table II.

C. Corpus Generation

Assume we have samples $\mathcal{S} \leftarrow \{S_1, S_2, S_3, \ldots, S_p\}$. Every sample S_i, $0 < i \leq p$, has thousands of unique features. A single sample S_i is reduced to a set of strings $S_i^r \leftarrow \{s_1, s_2, s_3, \ldots, s_m\}$ where s_i is a value of $f_i \in F$ and $m \leq k$. This feature reduction process is applied on all the samples in $\mathcal{S}$. This results in a set of reduced samples $\mathcal{S}^r \leftarrow \{S_1^r, S_2^r, S_3^r, \ldots, S_p^r, \}$. Define corpus as $\mathcal{C} \leftarrow \mathcal{S}^r$.

D. Vectorization

We generate vectors v_i from each sample $S_i \in \mathcal{C}$. After a set of vectors $V \leftarrow \{v_1, v_2, v_3, \}$ is generated, we construct a term frequency inverse document frequency matrix T. Every row within T represents a sample $S_i \in \mathcal{C}$ and every column represents a unique feature $f_i \in F$. Every cell (i, j) contains the frequency of a feature f_j found in sample S_i divided by the maximum frequency f_j found in the corpus $\mathcal{C}$. This factor helps to weigh how important a particular feature is within the corpus. Before T is produced, all features not found in at least 2 samples are dropped to reduce features which do not provide relational information. Once T is constructed, all values $t \in T$ are standardized[6]

E. Number of clusters

Some of the most popular clustering algorithms such as K-Means, require the number of clusters as input. However, in many cases, like malware clustering, the number of clusters for a set of data is unknown which limits potential clustering methods. To remedy this, we use gap statistic [14] to predict the number of clusters, g. Along with g, given that the ground-truth contains 94 clusters we construct a range of values k which begin at 47 and end at 141 in increments of 10 to represent a range between 0.5×94 to 1.5×94. Furthermore, we produce a set of 7 values that encapsulate a tight range around 94. Take the set K to be:

$$K \;=\; g \cup k \cup \{89, 92, 93, 94, 95, 96, 99\}$$

All clustering algorithms that assume the number of clusters, are tested on all values of K.

While we test only one well known method of calculating the number of clusters, we believe it is important to throughly test the viability of algorithms that require a known number of clusters. If favorable results are produced, many more methods for determining the number of clusters exist which could be tested in future work.

F. Similarity Construction

With T and V constructed, distance matrices M_k are generated which compute pair-wise dissimilarity of vectors $v_i \in V$. M_k is a matrix where each cell (i, j) has a floating point value representing the dissimilarity score of the pair of samples chosen, where a larger value denotes greater dissimilarity. Multiple methods were used in constructing the distance matrices. The first set of distance matrices were constructed using standard pairwise distance functions. Every pair of rows of TFIDF matrix T was run through several distance functions including Cosine[7], cityblock, Euclidean, Hamming, Braycurtis [15], Canberra [16], Minkowski, Chebyshev (L_∞ metric), and correlation distance functions, one distance matrix per function is constructed.

A second set of matrices was constructed directly from the document vectors V. Methods used are: Jaccard distance, Normalized Compression Distance, SSDeep Fuzzy Hashing, and several forms of Locality Sensitive Hashing. Jaccard index [17] is commonly used to compare the similarity between two sets. Jaccard distance is one minus the Jaccard index as shown:

$$Jaccard(x, y) \;=\; 1 \;-\; \frac{|\, x \;\cap\; y \,|}{|\, x \;\cup\; y \,|}$$

Jaccard was used in malware clustering works [8, 9, 11].

Normalized Compression Distance [18] is a way of computing distances between approximately similar instances and was used for malware clustering in [6]. It is defined as:

[6]Removing the mean and scaling to unit variance.

[7]To ensure that we get a proper metric, we compute the cosine distance as $1 - cos(d_1, d_2)$ where d_1 and d_2 are normalized document vectors.

$$NCD(x,y) \;=\; \frac{Z(\,x\;+\;y\,)\;-\;min(\,Z(x)\,,\;Z(y)\,)}{max(\,Z(x)\,,\;Z(y)\,)}$$

where $+$ is the concatenation operator and $Z(x)$ is the length of x after zlib compression.

SSDeep uses block based fuzzy hashing techniques [19] to produce fixed length hash blocks for certain amounts of input. These blocks can then be used to approximate similarity between two pieces of nonidentical but closely related data. Fuzzy hash distance computations have been used in malware clustering works [20].

Locality Sensitive Hashing [21] (LSH) is another hashing method for approximating similarity. It has been used in several malware clustering works, e.g., [7]. LSH is a method of hashing in which similar pieces of data have a higher chance of collision. By employing specific families of hash functions, distance metrics such as Jaccard can be approximated. This reduces CPU cycles and memory as distance between every pair of samples does not have to be calculated. Here we use min-hashing to approximate the Jaccard coefficient between samples. We do this by producing 5-gram character shingles for every sample vector $v_i \in V$. Fingerprints for each sample are then constructed by min-hashing these shingles.We implement two different methods of LSH for constructing distance matrices: (i) calculate the Jaccard distance between generated min-hash shingles of all samples determined to be matching pairs, called LSH Shingles. (ii) LSH Full - calculate the Jaccard distance between the complete feature vectors of every matched pair. The extra distance calculations between matching pairs for LSH Shingles & LSH Full reduced any incorrect approximations produced by LSH.

IV. Clustering Algorithms

Metamorphic malware is one that can transform based on the ability to translate, edit and rewrite its own code. In metamorphic malware, copies of the instances are functionally equivalent, but their internal structures and source codes differ. This ability allows new variants to evade detection. Furthermore, malware writers are aware of the Anti-Virus signature-based detection techniques and have implemented several methods to evade such signature-based detection [22] which include transformations such as register renaming, code permutation, dead code insertion and more.

A. Approach

To determine the families in which these metamorphic and polymorphic malware strands belong, several different clustering algorithms from a range of categories were employed including hierarchical, density based, prototype based, and dimensionality reducing methods. Each clustering algorithm is tested on a range of distance matrices along with variations of parameters values. We then evaluate all results against our ground-truth reference cluster using multiple metrics.

B. Density Clustering

To determine the quality of density based approaches for clustering malware we used the well known DBSCAN algorithm [23]. DBSCAN groups closely packed data together given distances. All points with a certain amount of near neighbors are converted into core points. In determining which points to be considered as near neighbors, a set of parameters D, where $D = \{0.25, 0.4, 0.5, 0.6, 0.75\}$, were tested as distance thresholds. Furthermore, in order to reduce the number of outliers generated, the number of neighbors necessary for construction of a core point was set to one. DBSCAN was then tested with every distance matrix constructed in combination with every distance threshold in D. Finally, we assigned all remaining unlabeled outliers into their own individual clusters. Vadrevu *et al.* [8] used DBSCAN for clustering malware.

C. Hierarchical Clustering

Hierarchical clustering is a method of clustering data through the construction of hierarchies. There are two main forms of hierarchical clustering, agglomerative and divisive. Her we focus on agglomerative. Agglomerative clustering builds hierarchies in a bottom up fashion with every sample initially starting out as a separate cluster. Through each step, nearest points are iteratively joined until all points are joined into one large cluster. A cutoff criterion is then used to determine where the hierarchy should be split to construct the clusters. In this work we test three different cutoff methods, by number of clusters, by max distance, and by Inconsistency measure.[8] When determining cutoff using number of clusters a result is produced for every value in the set K from section III-E. When selecting maximum distance cutoffs, the set of parameters $D = \{0.25, 0.4, 0.5, 0.6, 0.75\}$ used with DBSCAN IV-B, was tested. With the inconsistency metric we calculate the total inconsistency at every level. Any level with inconsistency value over the set threshold was where we place our cutoff. A set of parameters $I = \{0.75, 1.0, 1.25, 1.5, 1.75\}$ was tested for inconsistency thresholds. For constructing the hierarchies between clusters several different linkage methods were employed: single, complete, weighted, average, central, median, and ward linkages. These methods determine how distance is measured between hierarchies in order to combine them. All computed distance matrices were clustered using every linkage method to construct the cluster hierarchies, allowing us to see detailed changes which arise between distance metrics and forms of linkage. Then for every resulting hierarchy, several sets of clusters are extracted using the different cutoff methods (cluster count, max distance, and Inconsistency). Hierarchical clustering has been explored in previous work on malware clustering [6, 7, 9, 10].

[8] A link whose height differs noticeably from the height of the links below it indicates that the objects joined at this level in the cluster tree are much farther apart from each other than their components were when they were joined. This link is said to be inconsistent with the links below it. In cluster analysis, inconsistent links can indicate the border of a natural division in a data set.

D. Prototype Clustering

Many clustering algorithms encompass a prototype approach.[9] Here we implement two clustering algorithms that use prototypes for clustering. The first algorithm is affinity propagation [24], which takes as input measures of similarity between pairs of data points. Real-valued messages are exchanged between data points until a high-quality set of exemplars and corresponding clusters gradually emerges.

Our implementations of affinity propagation had a convergence number of 25 iterations and a iteration limit of 1000. Thus the algorithm would stop once no change had been seen for 25 iterations or a maximum of 1000 iterations was reached. We also applied a low damping value of 0.6, which gave us the most optimal results. With this implementation, we ran affinity propagation on the cosine, Jaccard, NCD, SSDeep, and LSH distance matrices.

The second method reliant on prototypes that we implemented was K-Means++ [25], a variant of the popular popular K-Means algorithm. Given a number of clusters k, K-Means initializes k random points as cluster centers. During every round, clusters are constructed by grouping all points to their closest cluster center via squared Euclidean distance. With clusters constructed, a new set of cluster centers are computed at the mean value of all points within a cluster, and clusters are recalculated. This iterative process continues until clusters have converged and no new results are produced from further rounds. However, results produced by K-Means can be highly sensitive to starting initializations of k. K-Means++ aims to mitigate this issue by assuming a more spread out set of cluster centers. While the first cluster center is randomly selected as one of the samples, all other points have a probability proportional to the squared Euclidean distance of its closest cluster center to be chosen as the next center. This allows points far from the closest center to have higher chances of being selected as cluster centers. The algorithm was run with all values in our constructed set K referenced in section III-E. As K-Means relies on the euclidean mean values of points for convergence, no distance matrices were used. Instead, the TFIDF matrix T was given to the algorithm which produced 'distances' in each iteration. Due to results varying between initializations, for every k value, 10 executions of the algorithm are processed and the cluster set with the best inertia [10] is returned as the result.

E. Dimensionality Reduction Clustering

Malware tend to produce large amounts of behavioral features. This can often lead to clustering points of high dimension. Due to the Curse of Dimensionality [26], distances tend to degrade within high dimensions which can be troubling for clustering.[11] Due to this we've also implemented forms of spectral clustering. Spectral clustering is useful in not only clustering high dimensional data, but is also useful for clustering non-convex data for which cluster centers are not good representatives. Given a similarity matrix, a normalized Laplacian matrix is constructed. To create clusters from this Laplacian, a popular choice is to use K-Means. However, due to K-Means being sensitive to initialization a Discretization [28] approach was also implemented which is less affected by starting conditions. Both K-Means and Discretize are run using this lower dimensional Laplacian on every value in K referenced in section III-E. For every run of K-Means and Discretize on a value $k \in K$, the algorithm is executed 10 times and the run with the best resulting inertia is returned as the result. Matrices for Cosine, Jaccard, NCD, SSDeep, and LSH metrics were used.

F. LSH Bins

Along with using LSH to construct distance matrices, we also test a method of clustering completely reliant on grouped pairs defined by LSH, we call this LSH Bins. Initially each sample is grouped as its own independent cluster. We then iteratively combine the clusters of every pair which was determined to be similar by LSH. This then produced a set of standalone clusters computed purely through LSH.

G. Quality Metrics

To determine cluster quality, eight different metrics were used to relate resulting clusters to our ground-truth, as well as one unsupervised verification metric.

One supervised metric we used was the Adjusted Mutual Information Score [29], a variant of the Mutual Information Score. Mutual Information calculates shared information between two sets of clusters. Adjusted Mutual Information takes into account chance, as two sets of large clusters tend towards a high MIS regardless of shared information.

Another metric used is the Adjusted Rand Score (ARS), a variant of Rand Index which also corrects for chance. Rand Index determines similarity between two sets of clusters by counting all pairs of samples assigned in similar and different clusters. The RI of a given set of clusters X and reference clusters Y containing n samples is:

$$RI(X,Y) \;=\; \frac{s \;+\; d}{s \;+\; d \;+\; x \;+\; y} \;=\; \frac{s \;+\; d}{\binom{n}{2}}$$

Where s is the number of sample pairs which are in similar clusters in X and in similar clusters in Y. d is the number of pairs which reside in different clusters in X and different clusters in Y. x is the number of pairs which are in the same cluster in X but belong to different clusters in Y. y represents

[9] This technique characterizes each cluster in terms of a cluster prototype; i.e., a data object that is representative of the other objects in the cluster such as centroid, medoid etc.

[10] Inertia in this case is the sum of the squared distances of all samples to their closest cluster center.

[11] In [27], it is shown, for certain data distributions, that the relative difference of the distances of the closest and farthest data points of an independently selected point goes to 0 as the dimensionality increases. Thus, it is often said, "in high dimensional spaces, distances between points become relatively uniform." In such cases, the notion of the nearest neighbor of a point is meaningless.

the number of pairs which are in different clusters in X but in the same cluster in Y. Note that the ARS gives a value in the range $[0.0, 1.0]$, the greater the score the more similar two sets of clusters.

Homogeneity, Completeness, and V-Measure [30] were also implemented as supervised verification metrics.

A given cluster set X has perfect Homogeneity if for every cluster $c \in X$, c only contains points which are clustered together within a given reference cluster set Y. Meaning that if all points represented independent clusters within X, X would produce perfect Homogeneity score regardless of Y.

In contrast, cluster set X would produce perfect Completeness if for every cluster $c \in Y$, every point within c would be grouped in the same cluster within X. Meaning that if all points were grouped in one large cluster within X, X would produce perfect Completeness score regardless of Y.

Therefore, a cluster set producing a high Homogeneity score but low Completeness represents a set of clusters which are more discriminative than the reference. While a set of clusters which attain low Homogeneity but high Completeness represents a system which returns more generalized results than that of the reference.

To combine Homogeneity and Completeness into one comprehensive score we use V-Measure. V-Measure is the harmonic mean of Homogeneity and Completeness. For a given cluster set X and reference clusters Y, V-Measure is given as:

$$V(X,Y) \;=\; 2 \;\times\; \frac{H(X,Y) \;\times\; C(X,Y)}{H(X,Y) \;+\; C(X,Y)}$$

Where $H(X,Y)$ is the Homogeneity score of X given Y and $C(X,Y)$ is the Completeness. Note that V-Measure is identical to the Normalized Mutual Information Score.

Two final supervised metrics used are Precision and Recall. Our definitions of Precision and Recall come from [7], and have been previously used as verification for malware clustering [7, 9, 10]. For a given set of reference clusters $Y \leftarrow \{Y_1, Y_2, \cdots, Y_r\}$ with r clusters and a set of clusters $X \leftarrow \{X_1, X_2, \cdots, X_c\}$ containing n samples split into c clusters, The Precision of X on Y is given as:

$$P(X,Y) \;=\; \frac{\sum_{i=1}^{c} \; max(\; |X_i \;\cap\; Y_1|\;,\; \cdots\;,\; |X_i \;\cap\; Y_r|\;)}{n}$$

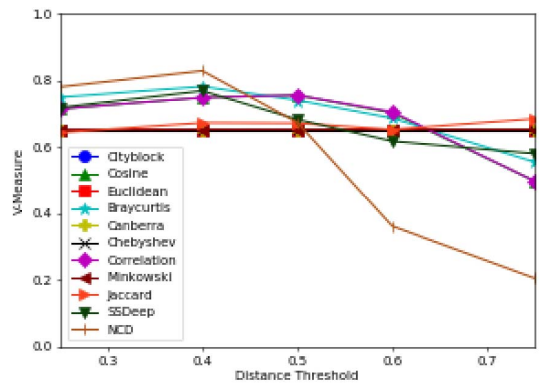
While Recall is given as:

$$R(X,Y) \;=\; \frac{\sum_{i=1}^{r} \; max(\; |Y_i \;\cap\; X_1|\;,\; \cdots\;,\; |Y_i \;\cap\; X_c|\;)}{n}$$

Precision and Recall behave similarly to Homogeneity and Completeness respectively. Meaning, Precision measures how well a system is able to discriminate different samples within the reference and Recall measures how effectively the system is able to recognize similar samples. We then use G-Mean as the geometric mean of Precision and Recall. G-Mean is given as:

$$G(X,Y) \;=\; \sqrt{P(X,Y) \;\times\; R(X,Y)}$$

Fig. 1. DBSCAN V-Measure across distance thresholds



Lastly, in order to evaluate clusters sets independently we use the Silhouette Coefficient [31]. The Silhouette Coefficient is a widely used cluster verification metric which determines how similar a point is to its own cluster compared with other clusters via a value between negative one to one. A Silhouette Coefficient of zero usually represents potentially overlapping clusters, a positive value indicates a sample is better represented in its own cluster than any other, and a negative value represents the opposite. Take a sample s, let the distance between s to a cluster C be the average of distances between s and all points $p \in C$. The Silhouette Coefficient of s is given as:

$$SC(s) \;=\; \frac{B(s) \;-\; A(s)}{max(\; A(s)\;,\; B(s)\;)}$$

Where $A(s)$ is the mean intra-cluster distance of all points to s and $B(s)$ is the distance between s and the closest non associated cluster. The value returned by A(s) can also be interpreted as how much a sample belongs to its cluster (smaller values means better fit). Unlike the other verification metrics, the Silhouette Coefficient does not require a reference cluster set and instead is only dependent on the distance values between the samples themselves. We define Silhouette Score as the mean Silhouette Coefficient of all samples, this provides an overall measure of quality.

While all metrics are able to outline the most effective cluster sets, some metrics are more sensitive to specific factors. For example Precision and Recall are generally sensitive to the number of clusters produced where as Homogeneity and Completeness are not. Furthermore, Adjusted Mutual Information Score and Adjusted Rand Index account for chance and thus are good metrics for large clusters. We implement these verification metrics in combination to allow for the most comprehensive overview of our results. The correlation between verification metrics calculated on http features is shown in Table IX.

TABLE III
TOP DBSCAN RESULTS

Distance	Measure	Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
0.4	NCD	945	**0.829**	0.819	0.836	**0.794**	**0.680**	**0.794**	0.896	0.703	0.360	1033.68
0.4	Braycurtis	1012	0.782	0.889	0.697	0.559	0.605	0.757	0.866	0.661	0.311	183.29
0.25	NCD	1702	0.781	0.966	0.549	0.768	0.521	0.728	0.966	0.549	0.328	1033.68

TABLE IV
TOP K-MEANS++ RESULTS

Measure	Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
Euclidean	5642	**0.651**	**1.0**	0.483	**0.267**	**0.207**	0.385	**0.999**	0.148	0.475	10,610.59
Euclidean	93	0.315	0.197	0.795	0.050	0.171	**0.564**	0.333	0.957	- 0.253	183.0
Euclidean	117	0.102	0.056	0.533	0.007	0.026	0.416	0.180	0.960	**0.496**	225.13

TABLE V
TOP SPECTRAL RESULTS

Version	Measure	Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
Discretize	Cosine	96	**0.893**	0.885	0.855	**0.870**	**0.864**	**0.870**	0.885	0.855	0.343	58.24
Discretize	Cosine	57	0.874	0.848	0.901	0.815	0.835	0.839	0.810	0.869	0.303	58.24
Discretize	Cosine	89	0.873	0.862	0.883	0.667	0.847	0.817	0.774	0.862	0.237	58.24

TABLE VI
TOP AFFINITY PROPAGATION RESULTS

Measure	Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
Cosine	1677	**0.374**	**0.551**	0.284	0	- 0.004	0.106	0.377	0.030	**0.441**	623.25
Jaccard	1181	0.319	0.428	0.255	- 0.002	- 0.003	**0.190**	0.311	0.116	0.232	607.39
SSDeep	1427	0.353	0.508	0.270	0	- 0.003	0.116	0.341	0.391	- 1.0	1121.89

Fig. 2. K-Means++ V-Measure and Silhouette Score Across K Values

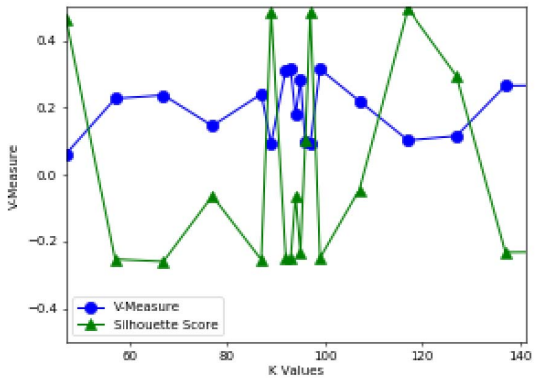
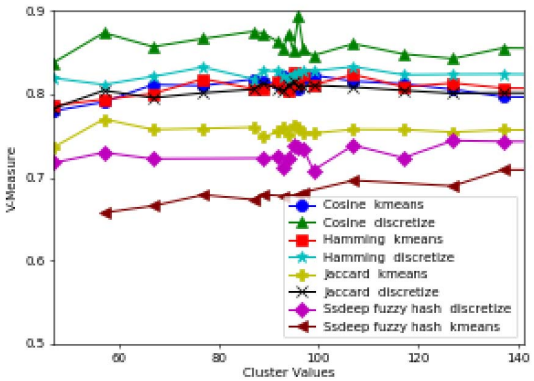


Fig. 3. Spectral V-Measure Across Cluster Values



V. RESULTS AND DISCUSSION

In the following sections we discuss performance specifications, best resulting cluster sets, and final evaluations.

A. Performance

While cluster quality is important to ensure the best groupings, cost and execution time is also a vital factor. With large amounts of new malware emerging daily, it is crucial to be able to process all this information in an efficient manner.

All computation was performed on a cluster using Intel Xeon E5-2699 v4 processors running at 2.20 GHz.

To improve timing and efficiency, all constructed distance matrices, with the exception of LSH, took advantage of full parallelization of 8 cores. When computing LSH, only the processing of hashes was parallelized.

Furthermore, computing clusters for DBSCAN and K-Means++ was also parallelized against 8 cores.

Recorded timings are presented alongside the best results for each clustering algorithm.

TABLE VII
TOP HIERARCHICAL RESULTS FOR EACH CUTOFF METHOD

Cutoff	Linkage	Measure	Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
Cluster Value	Average	Braycurtis	107	**0.921**	0.911	0.916	0.878	0.901	0.889	0.863	0.916	0.407	195.71
Dist. = 0.75	Average	Braycurtis	525	0.870	0.971	0.788	0.846	0.742	0.834	0.948	0.733	0.464	195.71
Incon. = 1.75	Complete	Jaccard	566	0.790	0.966	0.667	0.592	0.584	0.680	0.951	0.486	0.286	90.90

TABLE VIII
LSH BINS RESULT

Clusters	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.	Time (sec)
277	0.326	0.208	0.748	0.033	0.151	0.507	0.273	0.941	-1.0	135.67

Fig. 4. Hierarchical V-Measure Across Distance Thresholds

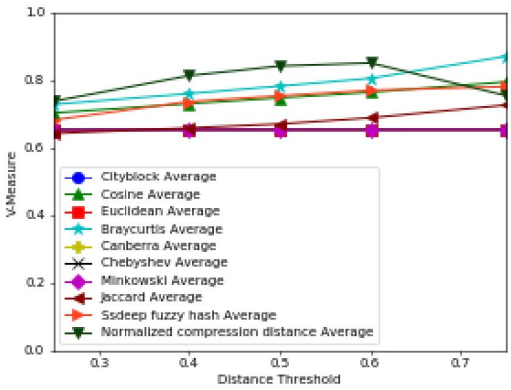


Fig. 5. Average V-Measure of Hierarchical Linkages

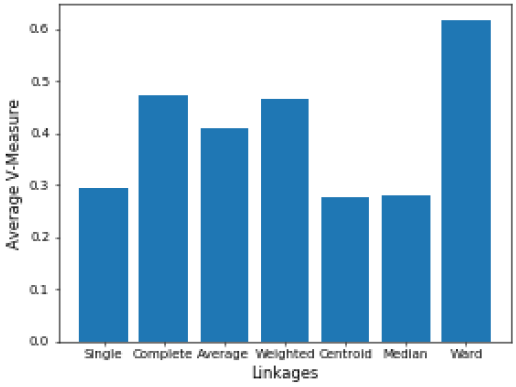
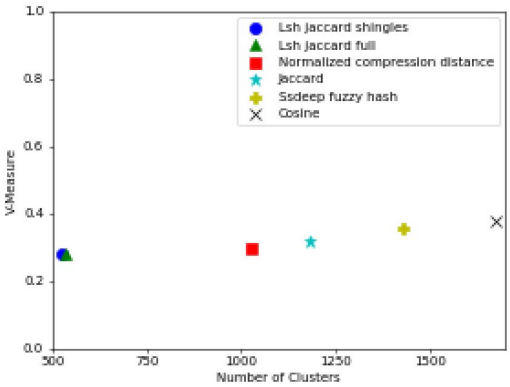


Fig. 6. Affinity Propagation V-Measure Across Cluster Values



B. Results

Running all clustering methods yielded 3,055 different results per feature combination used.

Initially clusters were tested using four sets of features, behavioral, network, cuckoo signature, and all features in combination. Of all the results network features demonstrated the best performance while all features combined presented the worst.

To determine the most prominent features within the Network category, we ran tests using a finer subsets of network data. Several independent network feature categories were initially tested along with grouping of several well performing features. The list of combinations are: (a) HTTP (b) UDP & TCP (c) DNS (d) Domains (e) Hosts (f) ICMP (g) IRC (h) SMTP (i) HTTP & UDP & TCP (j) HTTP & DNS & Domains & Hosts (i) UDP & TCP & DNS & Domains & Hosts.

From the results, we observe that the top quality clusters in all aspects were based on HTTP features alone followed by all network features. This result is a *strong* validation of the input dataset we had. The clustering algorithms were oblivious to the fact that the input dataset were purely NIDS-based.[12] As HTTP features gave the best results across the board, we use this feature set when representing our statistics. Furthermore, the highest scores for each clustering algorithm are set in bold for clarity. However, not all results presented within the tables have the highest scores for every metric. Thus, some columns may not contain any bold values.

The HTTP category contained six features in total that consisted of information from the URI of the request including the host, path (location for which the request is being sent), User-Agent, and the entire URI. The other 2 consisted of

[12]Network Intrusion Detection System

TABLE IX
METRIC CORRELATION MEASURES ON HTTP CLUSTERING RESULTS

Metric	VM.	Homo.	Comp.	Adj RI	Adj MIS	GMean	Prec.	Rec.	Sil.
VM	1	0.942	0.274	0.826	0.818	0.692	0.922	-0.566	0.131
Homo.	0.942	1	-0.033	0.692	0.608	0.434	0.997	-0.792	0.199
Comp.	0.274	-0.033	1	0.530	0.691	0.813	-0.080	0.603	-0.146
Adj RI	0.826	0.692	0.530	1	0.927	0.865	0.663	-0.156	0.208
Adj MIS	0.818	0.608	0.691	0.927	1	0.957	0.568	-0.028	0.108
GMean	0.692	0.434	0.813	0.865	0.957	1	0.394	0.188	0.055
Prec.	0.922	0.997	-0.080	0.663	0.568	0.394	1	-0.816	0.214
Rec.	-0.566	-0.792	0.603	-0.156	-0.028	0.188	-0.816	1	-0.170
Sil.	0.131	0.199	-0.146	0.208	0.108	0.055	0.214	-0.170	1

information being transmitted within the request including the body and data of only HTTP traffic (not HTTPS).

The Gap Statistic we calculated using these HTTP features was 5,642, which unfortunately was nowhere near our ground-truth of 94 clusters.

1) DBSCAN: The top results for DBSCAN are presented in Table III. NCD run on a distance threshold of 0.4 presented the best results in almost all metrics. Several interesting pieces of information can also be gleaned from Figure 1, representing the change in V-Measure across different distance thresholds for each similarity metric. While some metrics such as City-block, Cosine, Canberra, and Chebyshev produce almost no noteworthy change across different distance thresholds, a clear downward trend peaking near thresholds of 0.4 and 0.5 can be seen from Braycurtis, Correlation, Euclidean, and NCD with NCD being the most drastic. An interesting observation seen with Jaccard's behavior is that it seems to rise at thresholds of 0.4 and 0.5, fall at a threshold of 0.6, and rise again even higher at a threshold of 0.75. Distances produced via Hamming and LSH gave such low V-Measure scores that they were not identifiable on this graph and thus were not included.

2) K-Means++: Overall K-Means++ did not produce favorable clustering results as shown in Table IV. One interesting observation is that the calculated Gap-Statistic of 5,642 produced the highest V-Measure when used as K despite being the farthest value from the number of clusters in the ground-truth. Figure 2, which presents V-Measure and Silhouette score for a range of k values, shows significant fluctuation in results of both V-Measure and Silhouette score on k's near the number of ground-truth clusters. To maintain readability the gap statistic was not included.

3) Affinity Propagation: The top results returned by Affinity Propagation presented in Table VI, gave poor quality. As seen in Figure 6, the number of clusters generated by each run was significantly higher than that of our ground-truth.

4) Spectral: Spectral clustering produced the second most favorable results. The top three results shown in Table V, all result from the Cosine metric. The discretized implementation for each distance metric performed better overall than their K-Means counterpart. This can be seen in Figure 3, which presents V-Measure across a range of cluster values for the best performing distance metrics.

5) Hierarchical: Hierarchical clustering produced the best results. Scores for the top results produces using each cutoff method can be seen in Table VII. While the best results were produced using the number of cluster values as a cutoff, clusters produced using distance thresholds also gave strong results which rivaled the best performing results from Spectral clustering. Results of average V-Measure for each linkage is shown in Figure 5. While average, weighted, and complete linkages produced the highest performing results, ward linkage gave the best V-Measure on average. V-Measure for the best performing distance metrics combined with average linkage are shown along several distance thresholds in Figure 4.

6) LSH Bins: While LSH Bins was highly cost effective, it did not yield optimal results as can be seen in Table VIII.

One idea we experimented with was the notion that Silhouette score could be used to determine if a resulting cluster set was optimal without the use of reference clusters. However, as shown in Table IX, Silhouette score did not present a high correlation with other supervised verification metrics.

Hierarchical clustering, Spectral clustering, as well as DBSCAN yielded the best results. While our highest scoring implementations of Spectral and Hierarchical clustering required prior knowledge of the number of clusters, implementations of DBSCAN and other Hierarchical methods were also able to produce effective results without this assumption.

VI. CONCLUSIONS AND FUTURE WORK

In this paper, we have systematically and comprehensively studied the malware clustering problem with real-world malware data and a reference clustering from domain experts. Inspired by the rapid growth in the importance of cybersecurity and the need to address the scaling issues it poses, we plan to take this study further in several ways. In addition, this study leaves several questions for future work. First, similar to affinity propagation and spectral clustering, kernel methods can be useful to cluster features. We hope to conduct experiments to see which approaches lend well to bounded non-linear decision spaces. Second, we plan to see if can parallelize these algorithms and run them in a distributed system such as Hadoop/Spark to speedup the computation time.

ACKNOWLEDGEMENTS

This research was supported in part by NSF grants CNS 1319212, DGE 1433817, and DUE 1356705. This material is also based upon work supported by, or in part by, the U. S. Army Research Laboratory and the U. S. Army Research

Office under contract/grant number W911NF-16-1-0422. We also thank the University of Houston for their SURF grant.

References

[1] "Riskiq," https://www.riskiq.com/infographic/evil-internet-minute/, accessed: 2018-08-21.

[2] F. Abolhassan, *Cyber Security. Simply. Make It Happen.: Leveraging Digitization Through IT Security*, 1st ed. Springer Publishing Company, Incorporated, 2017.

[3] "Avtest," https://www.av-test.org/en/statistics/malware/, accessed: 2018-08-17.

[4] "Infosec," https://www.infosecurity-magazine.com/news/360k-new-malware-samples-every-day/, accessed: 2018-08-17.

[5] "Kaspersky," https://www.kaspersky.com/about/press-releases/2017_kaspersky-lab-detects-360000-new-malicious-files-daily, accessed: 2018-08-17.

[6] M. Bailey, J. Oberheide, J. Andersen, Z. M. Mao, F. Jahanian, and J. Nazario, "Automated classification and analysis of internet malware," in *International Workshop on Recent Advances in Intrusion Detection*. Springer, 2007, pp. 178–197.

[7] U. Bayer, P. M. Comparetti, C. Hlauschek, C. Kruegel, and E. Kirda, "Scalable, behavior-based malware clustering." in *NDSS*, vol. 9. Citeseer, 2009, pp. 8–11.

[8] P. Vadrevu and R. Perdisci, "Maxs: Scaling malware execution with sequential multi-hypothesis testing," in *Proc. 11th ACM on Asia Conference on Computer and Communications Security*. ACM, 2016, pp. 771–782.

[9] A. Mohaisen, O. Alrawi, and M. Mohaisen, "Amal: High-fidelity, behavior-based automated malware analysis and classification," *computers & security*, vol. 52, pp. 251–266, 2015.

[10] K. Rieck, P. Trinius, C. Willems, and T. Holz, "Automatic analysis of malware behavior using machine learning," *Journal of Computer Security*, vol. 19, no. 4, pp. 639–668, 2011.

[11] M. Polino, A. Scorti, F. Maggi, and S. Zanero, "Jackdaw: Towards automatic reverse engineering of large datasets of binaries," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2015, pp. 121–143.

[12] P. Li, L. Liu, D. Gao, and M. K. Reiter, "On challenges in evaluating malware clustering," in *International Workshop on Recent Advances in Intrusion Detection*. Springer, 2010, pp. 238–255.

[13] "Cuckoo sandbox," https://cuckoosandbox.org, accessed: 2018-08-21.

[14] R. Tibshirani, W. Guenther, and T. Hastie., "Estimating the number of clusters in a data set via the gap statistic," *Journal of the Royal Statistical Society Series B*, 2001.

[15] J. R. Bray and J. T. Curtis, "An ordination of the upland forest communities of southern wisconsin," *Ecological monographs*, vol. 27, no. 4, pp. 325–349, 1957.

[16] G. N. Lance and W. T. Williams, "Computer programs for hierarchical polythetic classification (similarity analyses)," *The Computer Journal*, vol. 9, no. 1, pp. 60–64, 1966.

[17] A. Z. Broder, S. C. Glassman, M. S. Manasse, and G. Zweig, "Syntactic clustering of the web," *Computer Networks and ISDN Systems*, vol. 29, no. 8, pp. 1157–1166, 1997.

[18] M. Li, X. Chen, X. Li, B. Ma, and P. M. B. Vitanyi, "The similarity metric," *IEEE Transactions on Information Theory*, vol. 50, no. 12, pp. 3250–3264, Dec 2004.

[19] J. Kornblum, "Identifying almost identical files using context triggered piecewise hashing," *Digital investigation*, vol. 3, pp. 91–97, 2006.

[20] M. Graziano, D. Canali, L. Bilge, A. Lanzi, and D. Balzarotti, "Needles in a haystack: mining information from public dynamic analysis sandboxes for malware intelligence," in *USENIX Security Symposium*. USENIX Association, 2015, pp. 1057–1072.

[21] A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*. New York, NY: Cambridge U. Press, 2011.

[22] P. Szor, *The Art of Computer Virus Research and Defense*. Addison-Wesley Professional, 2005.

[23] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise." in *Kdd*, vol. 96, no. 34, 1996, pp. 226–231.

[24] B. J. Frey and D. Dueck, "Clustering by passing messages between data points," *Science*, vol. 315, pp. 972–976, 2007.

[25] D. Arthur and S. Vassilvitskii, "K-means++: The advantages of careful seeding," in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '07, 2007, pp. 1027–1035.

[26] R. BELLMAN, *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961. [Online]. Available: http://www.jstor.org/stable/j.ctt183ph6v

[27] K. S. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is "nearest neighbor" meaningful?" in *Proc. 7th Int'l Conf. on Database Theory*, ser. ICDT '99, 1999, pp. 217–235.

[28] X. Y. Stella and J. Shi, "Multiclass spectral clustering," in *null*. IEEE, 2003, p. 313.

[29] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *J. Mach. Learn. Res.*, vol. 11, pp. 2837–2854, Dec. 2010.

[30] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proc. EMNLP-CoNLL*, 2007.

[31] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of computational and applied mathematics*, vol. 20, pp. 53–65, 1987.